



ML FRIDAYS

Deployment of LLMs using Amazon SageMaker

Sudhanshu Hate

Principal Architect,
Amazon Web Services

Agenda

- Large model hosting challenges
- Model optimization techniques
- Large model inference patterns with Amazon SageMaker
- Performance best practices

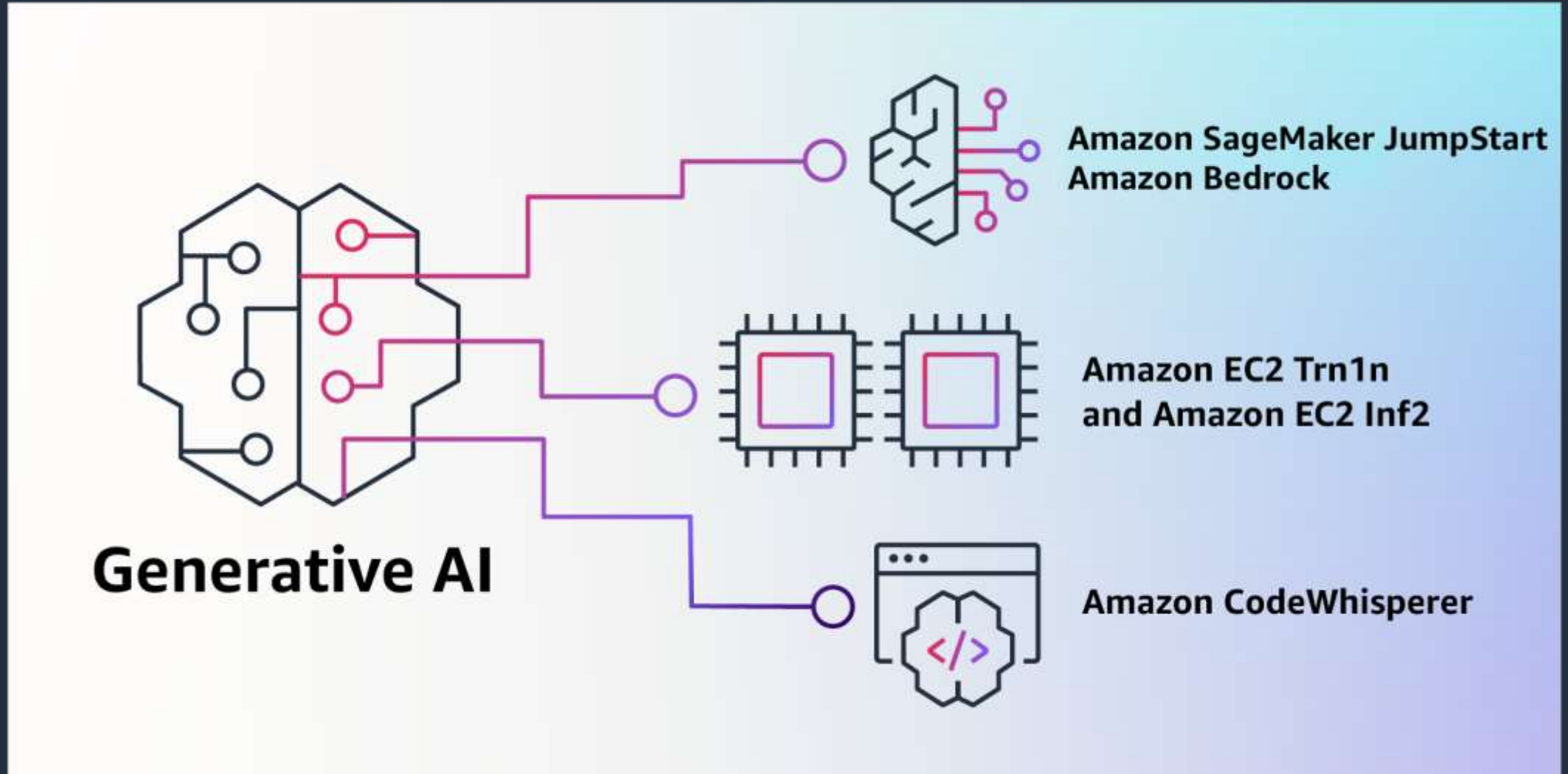


Prerequisites

- Understanding of LLMs
- Understanding of Amazon Sagemaker Model training



Building with generative AI on AWS



Selection of pre-trained models to suit your use case

Publicly available

Proprietary models

Stability AI

Models

- Text2Image
- Upscaling

Tasks

- Generate photo-realistic images from text input
- Improve quality of generated images

Features

- Fine-tuning on SD 2.1 model

AlexaTM

Models

- AlexaTM 20B

Tasks

- Machine translation
- Question answering
- Summarization
- Annotation
- Data generation

Hugging Face

Models

- Bloom 1b1
- Bloom 1B7
- Bloom 560m
- GPT2
- DistilGPT2

Tasks

- Generate photo-realistic images from text input
- Improve quality of generated images

Co:here

Models

- Cohere generate Model - medium

Tasks

- Text generation
- Information extraction
- Question answering
- Summarization

LightOn

Models

- Lyra-Fr 10B

Tasks

- Text Generation
- Keyword extraction
- Information extraction
- Question answering
- Summarization
- Sentiment analysis
- Classification

AI21 Labs

Models

- Jurassic-1 Grande 17B

Tasks

- Text generation
- Long-form generation
- Summarization
- Paraphrasing
- Chat
- Information extraction
- Question answering
- Classification



Amazon SageMaker - overview



Amazon SageMaker

SAGEMAKER STUDIO IDE

Real-time inference Async inference Serverless inference Batch inference Multi-model endpoints Multi-container endpoints Inference pipelines Manage and version models MLOps Model monitoring Shadow Testing Metrics and logging in CloudWatch

SageMaker JumpStart

CONTAINERS



ML COMPUTE INSTANCES & ACCELERATORS

CPU	GPU	AWS Inferentia2 & AWS Trainium	Graviton (Arm)
-----	-----	--------------------------------	----------------

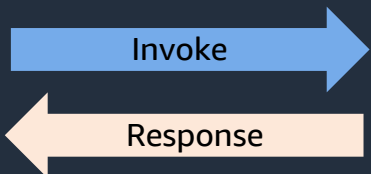
DEEP LEARNING COMPILERS AND RUNTIMES

SageMaker Neo	NVIDIA TensorRT/cuDNN	Intel oneDNN	Arm Compute Library
---------------	-----------------------	--------------	---------------------

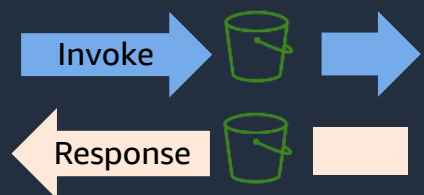


Deploy model to serve inference

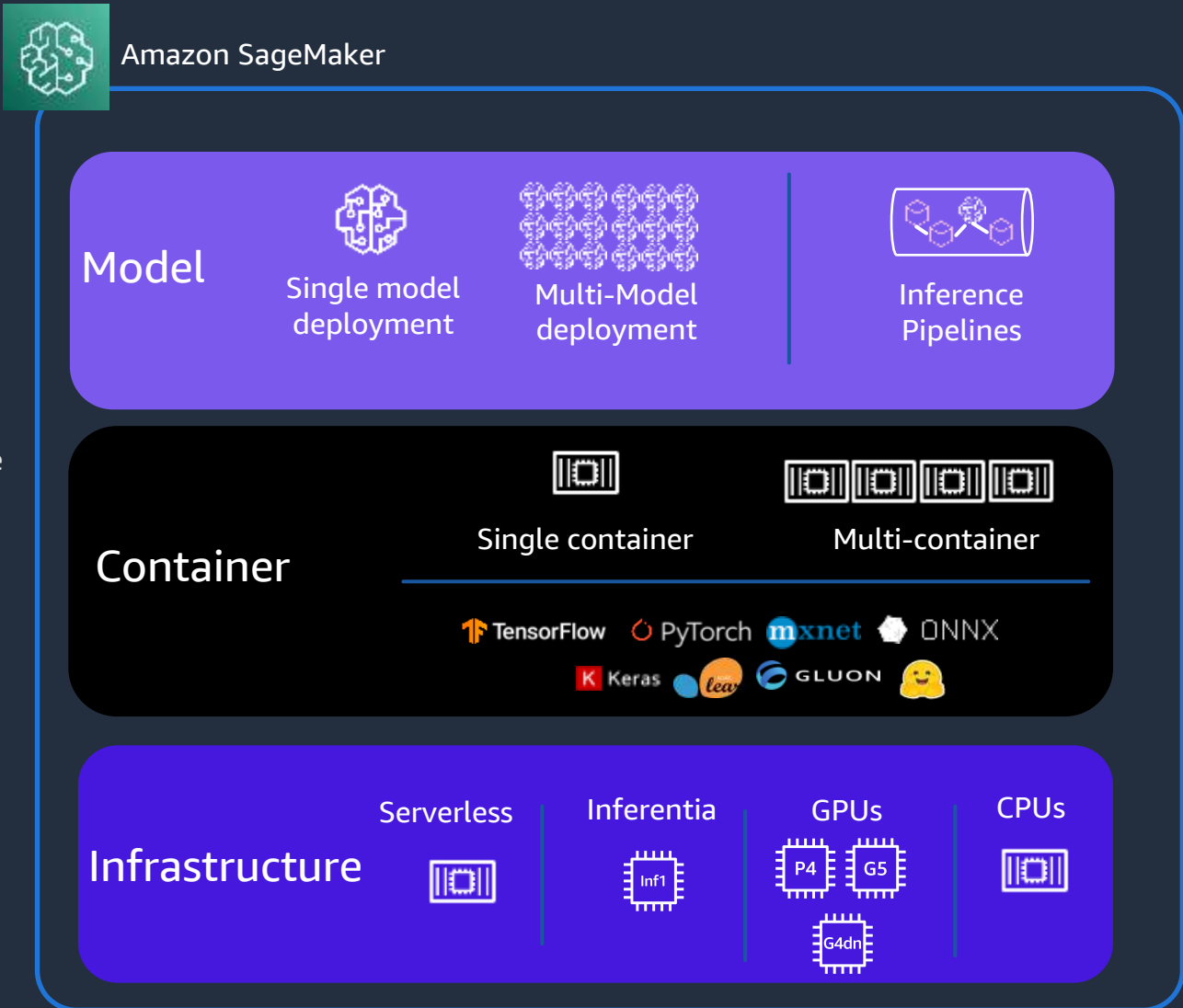
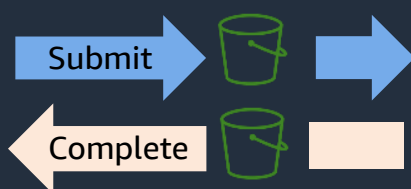
- Real-time synchronous response



- Near real-time asynchronous response



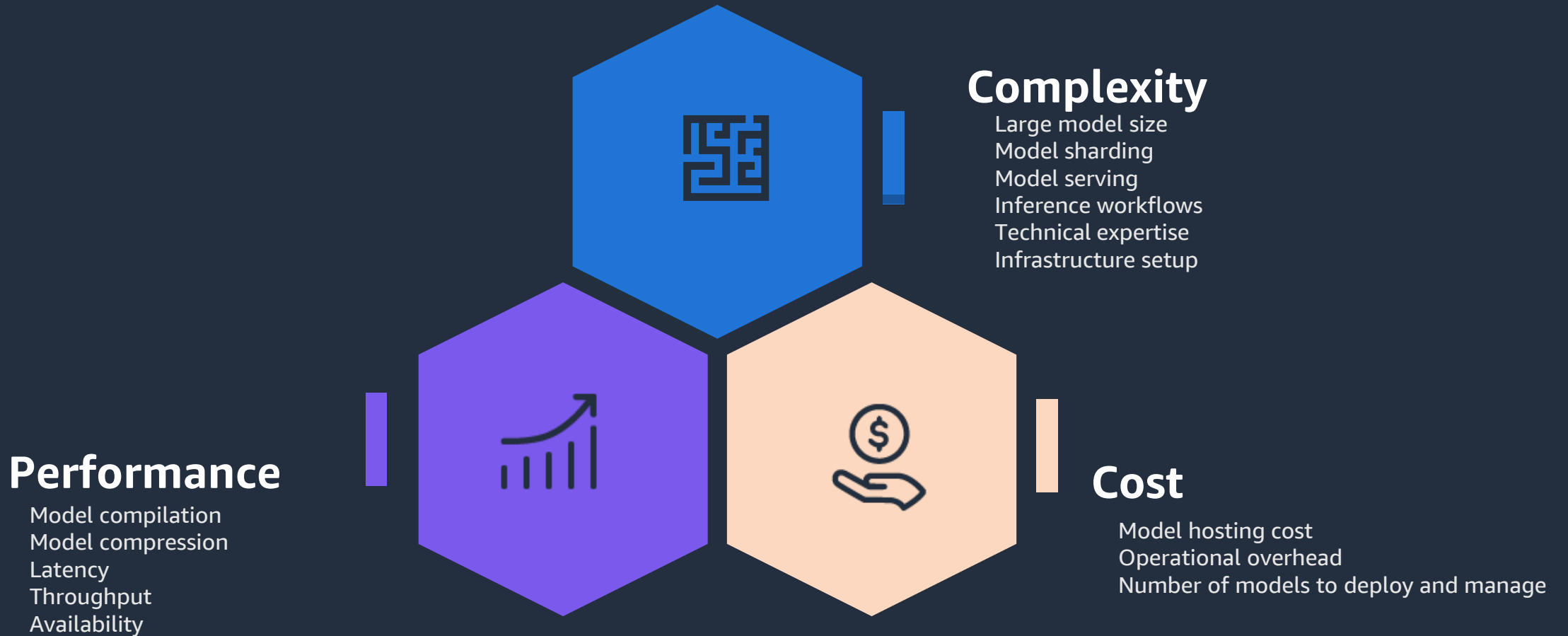
- Offline batch inference



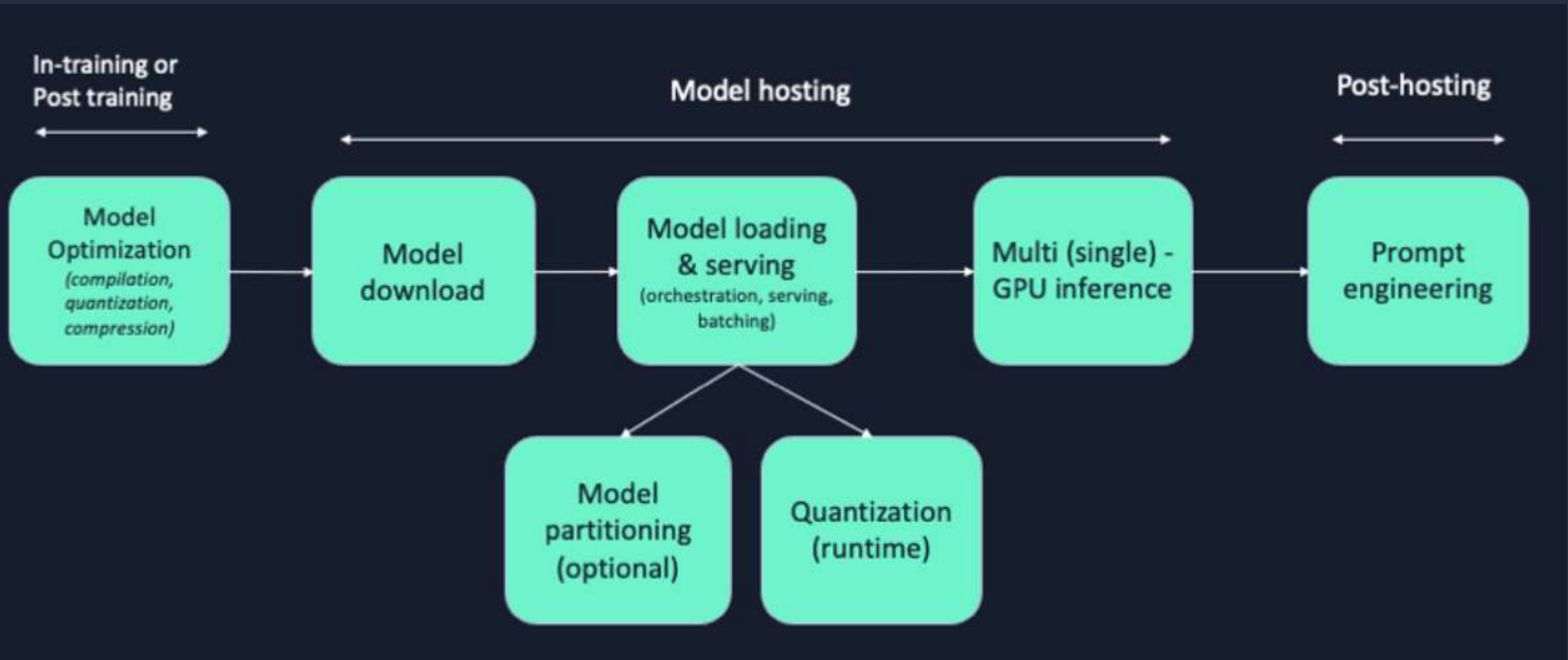
Large model hosting challenges



Large model hosting challenges

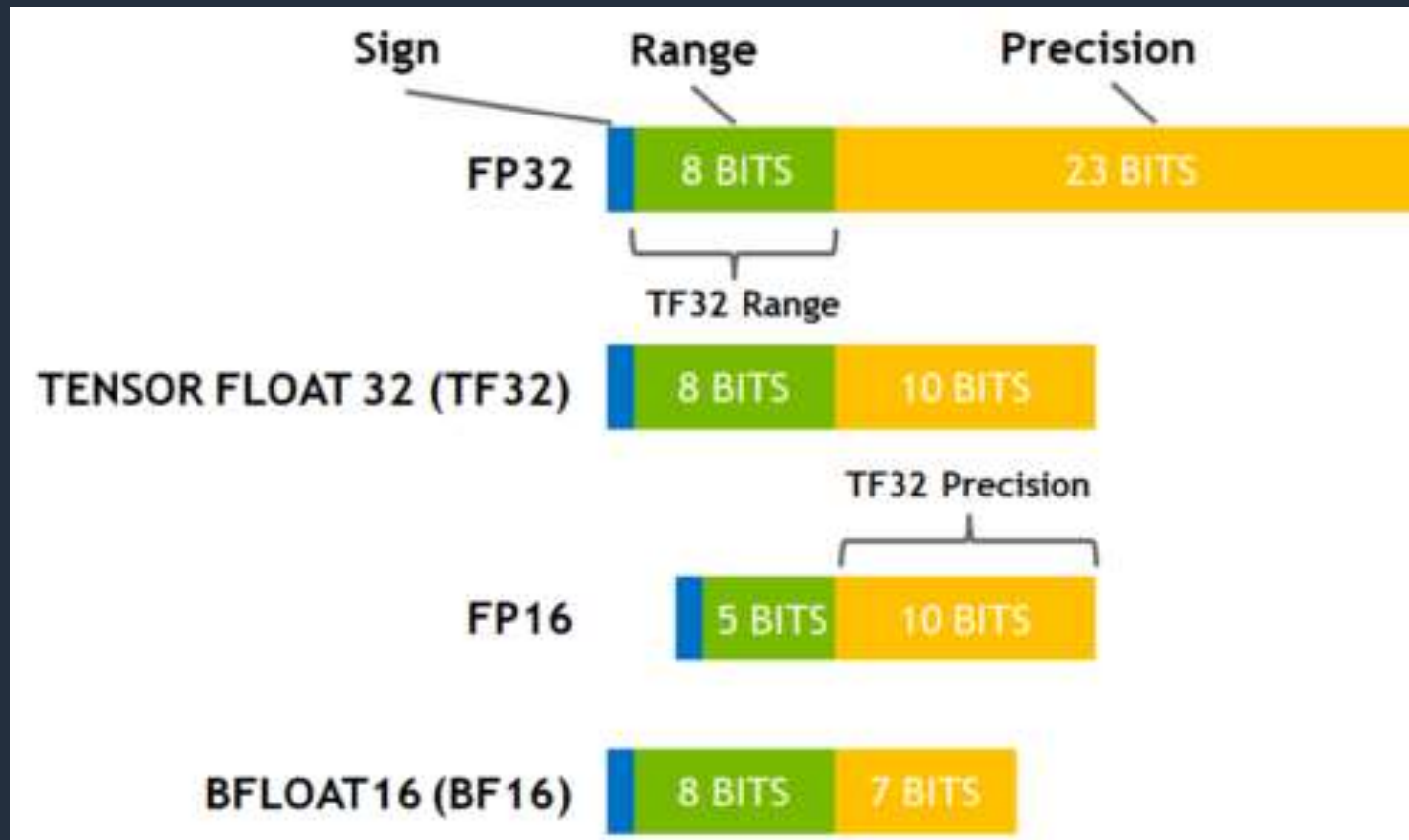


Large Model Hosting



Model optimization techniques

Precisions - background



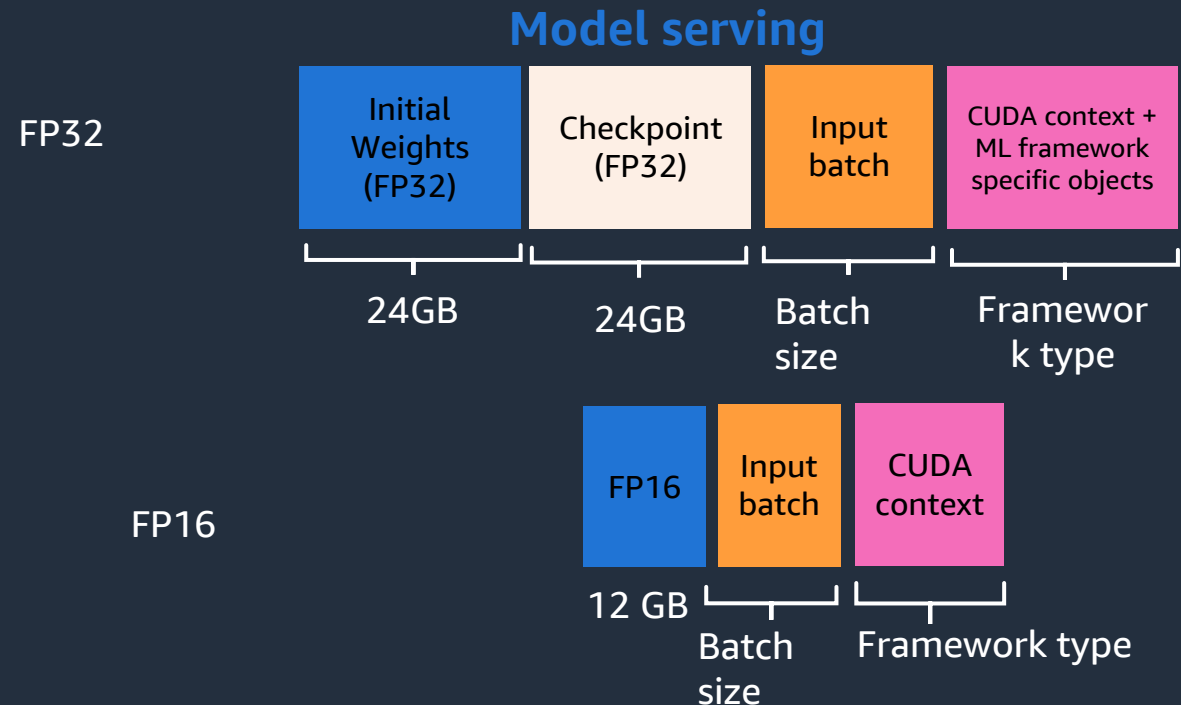
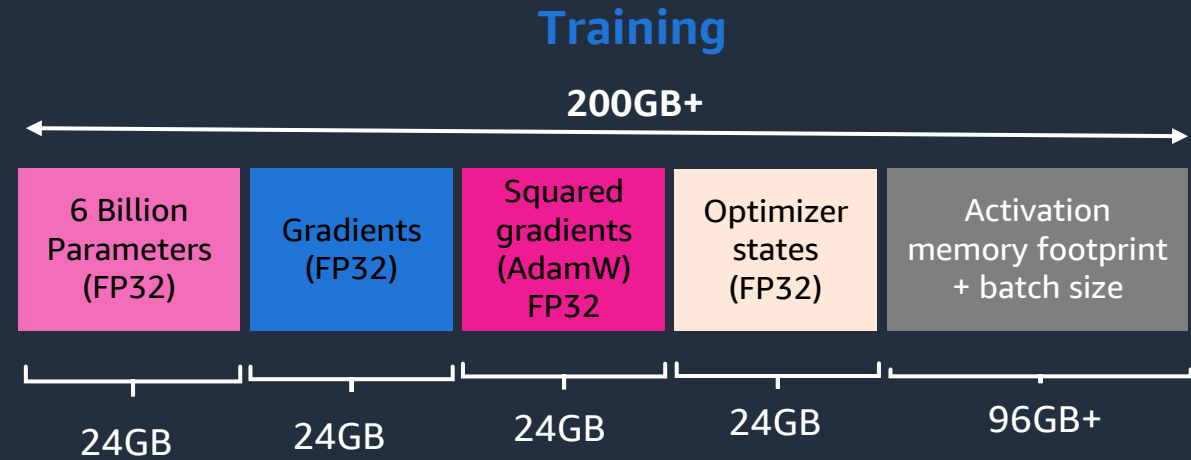
Model size

GPT- J 6-Billion

Memory required to load model weights for FP32
= 4 bytes x number of parameters
(6-billion) = **24 GB**

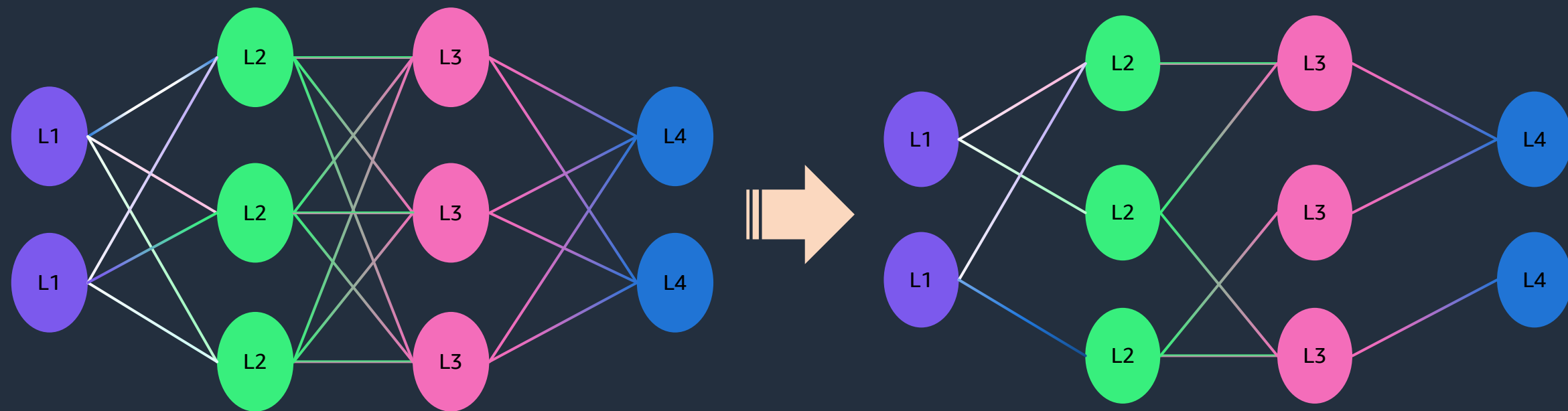
Float16 will speed up the model by ~20% and reduce memory consumption by 2x.

Size comparison:
FP 16 = ½ FP 32
BF 16 = ½ FP 32
INT 8 = ¼ FP 32



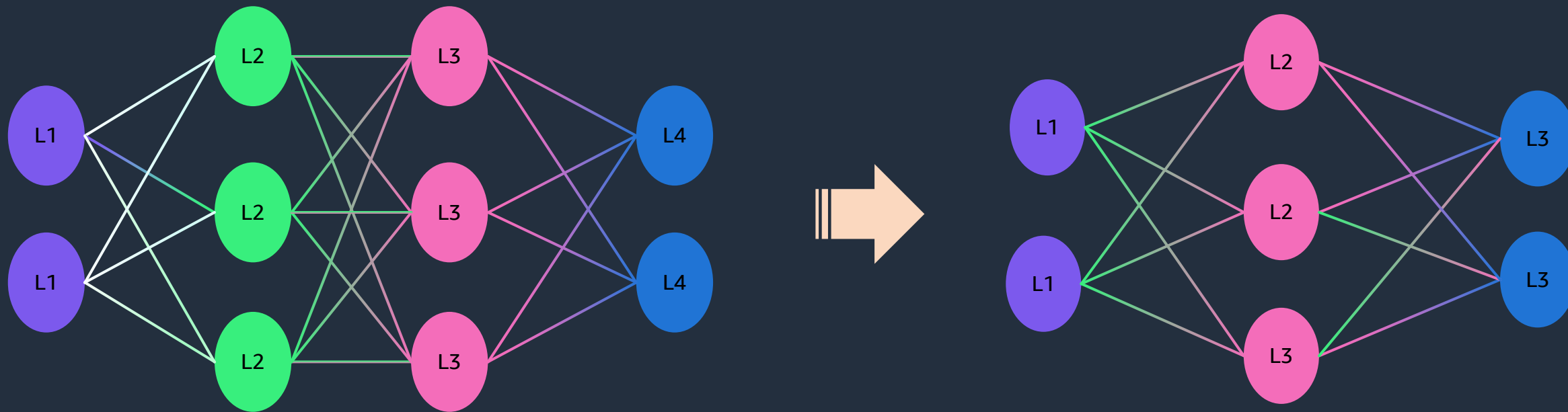
Model compression

PRUNING



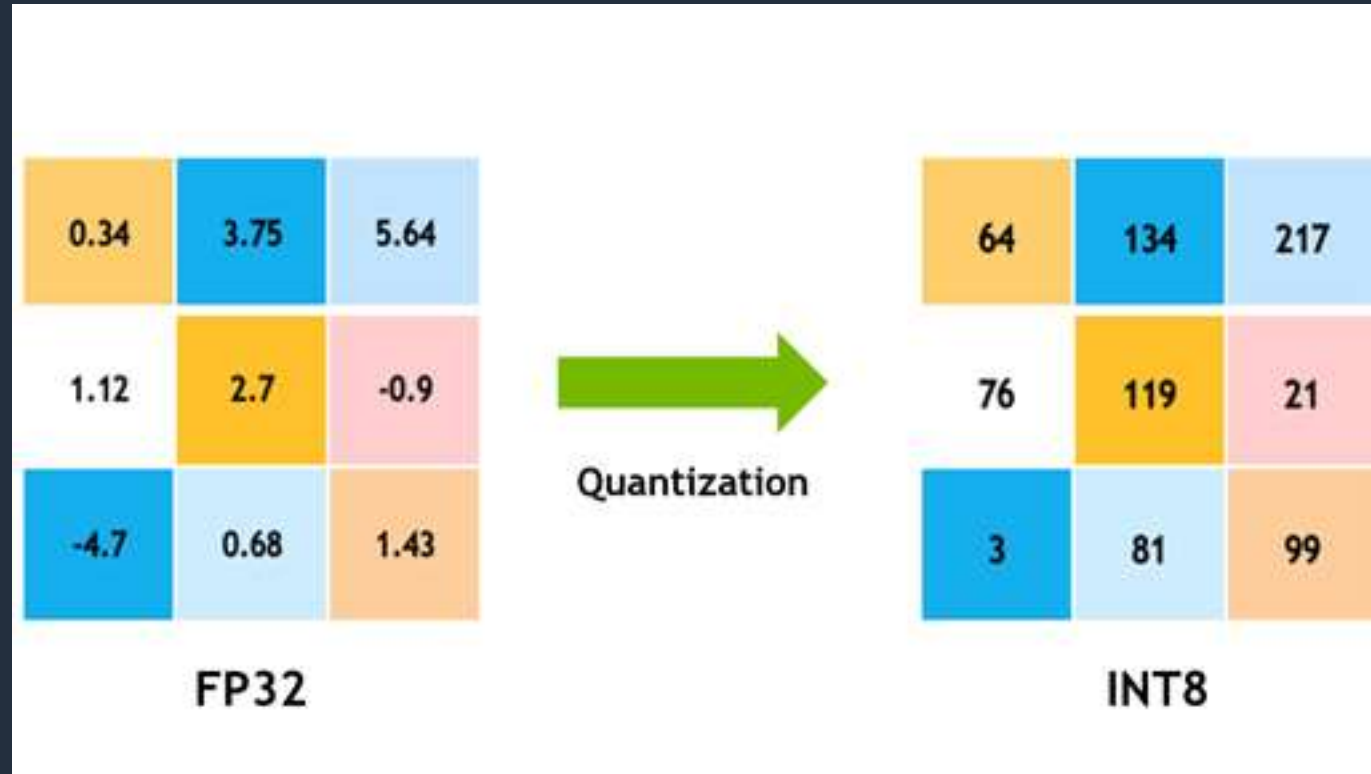
Model compression

KNOWLEDGE DISTILLATION



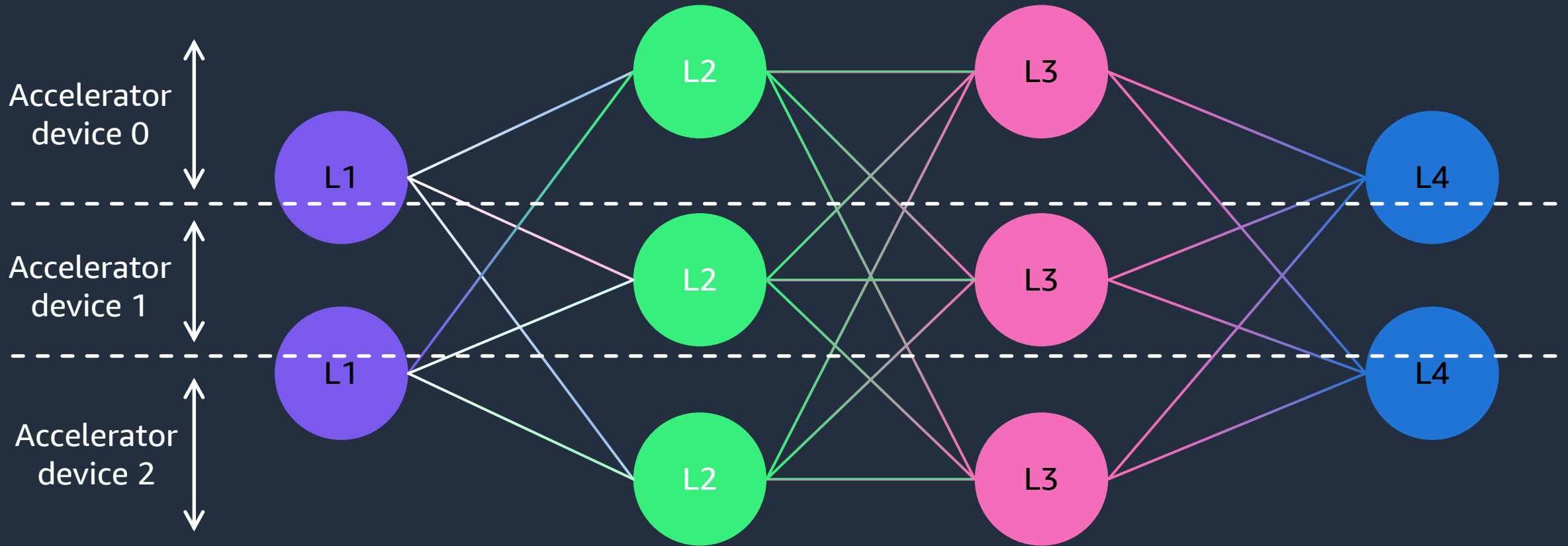
Model compression

QUANTIZATION



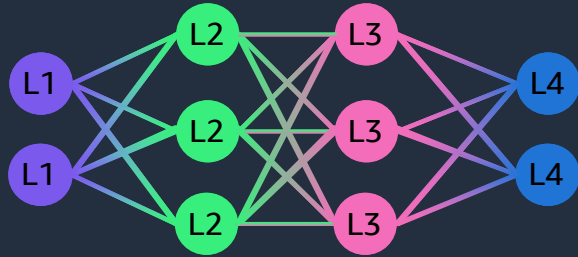
Tensor parallel model serving

ALSO CALLED INTRA-LAYER MODEL PARALLELISM

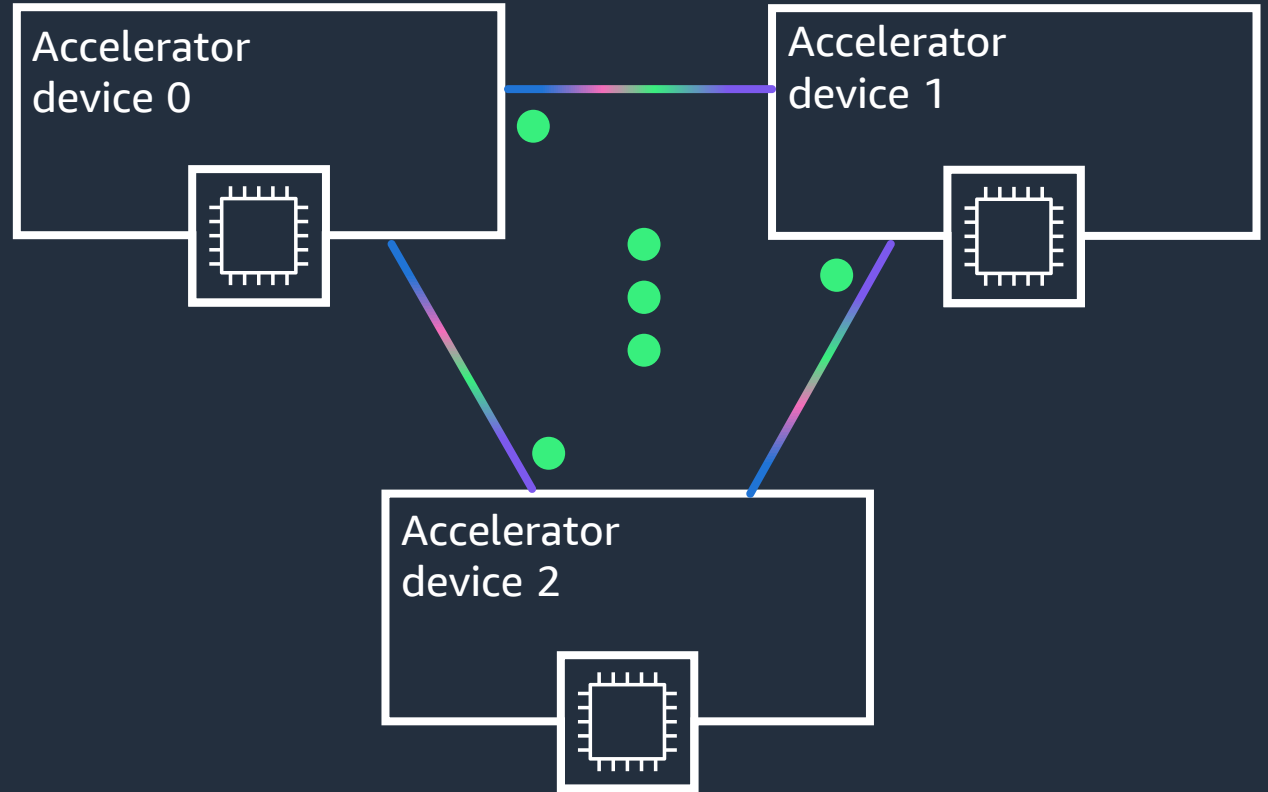


Tensor parallel (horizontal) model serving

ML
model

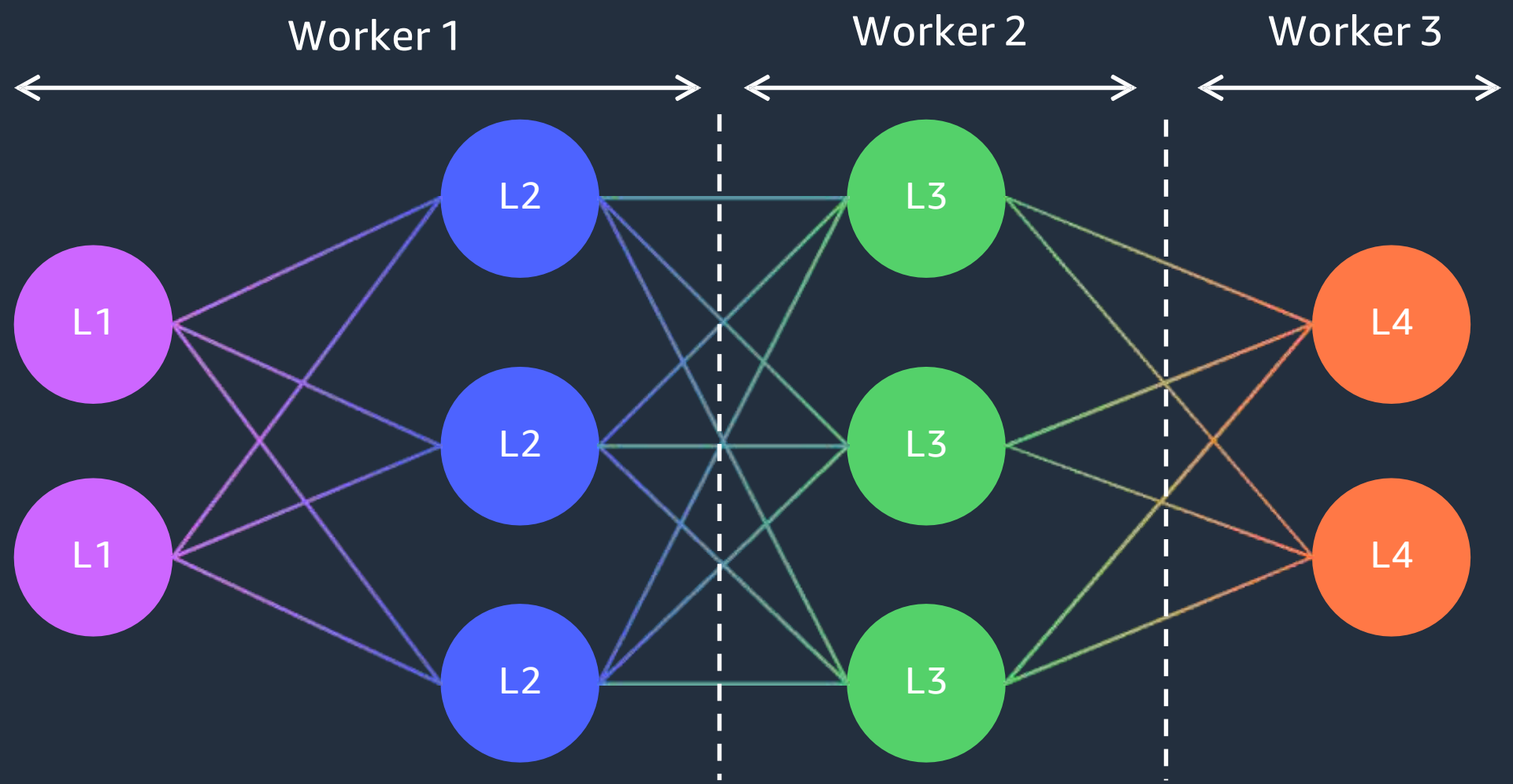


Inference
data



Pipeline (vertical) parallelism

*ALSO CALLED INTER-LAYER MODEL PARALLELISM



Which partitioning approach to choose for large model inference

Tensor parallel (TP)

- Fits larger models and layers
- Better in scaling efficiency
- All-to-all communication

Pipeline parallel

- Fits larger models
- P2P communication
- What if the layer does not fit on single accelerator device?
- Pipeline idleness reduces scaling efficiency

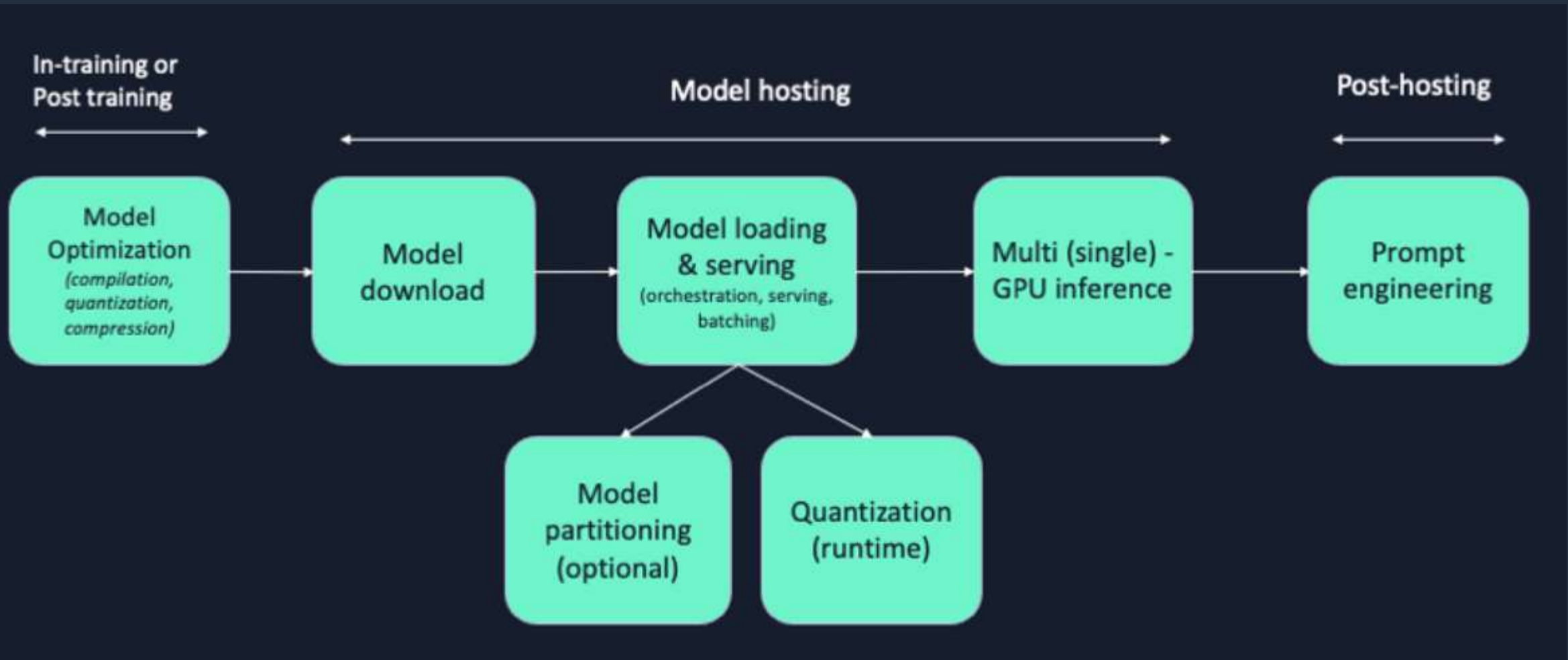
We recommend to try TP before trying pipeline parallel



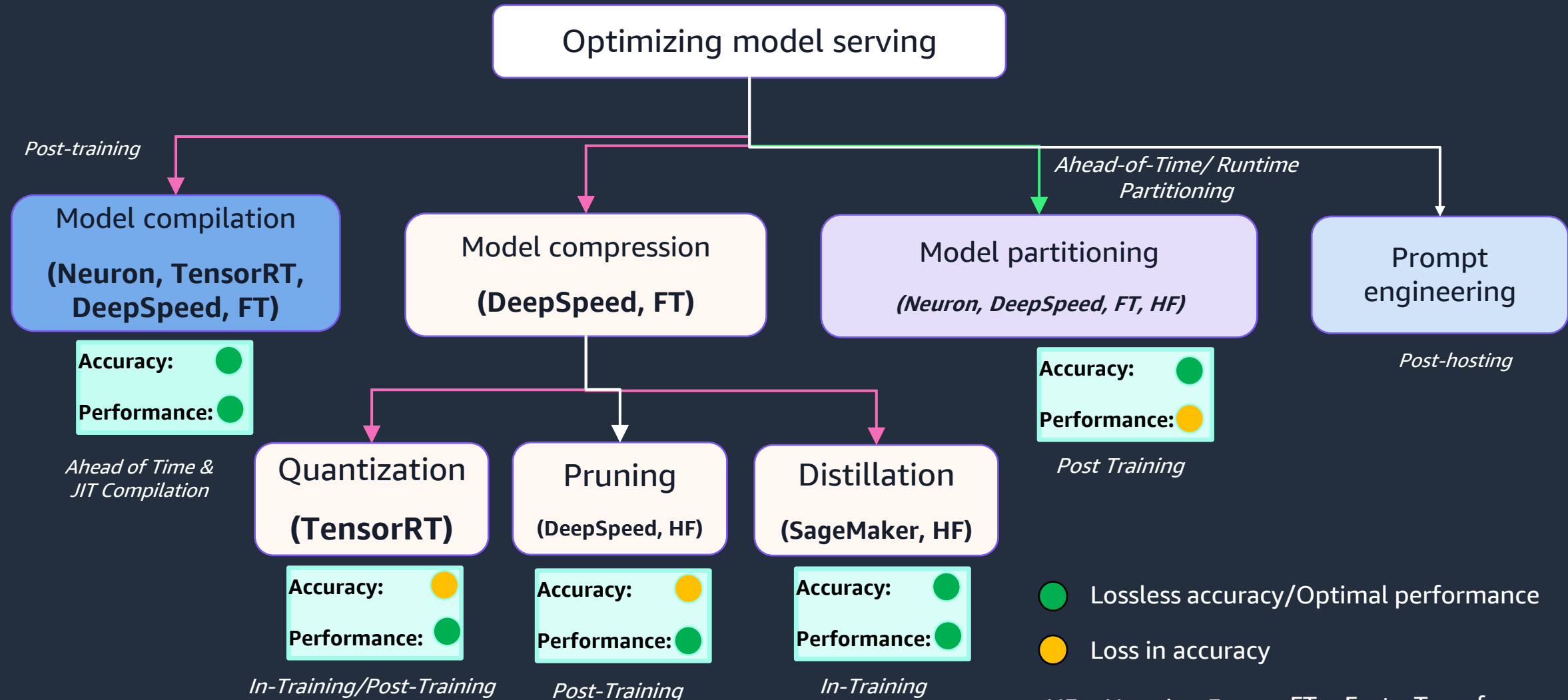
Model Compression Techniques

	Description	Accuracy	Inference latency	Memory consumption	User effort/Cost
Post-training quantization (PTQ)	Reduce precision on model weights & activations.	Medium	Low	Low	Low
Quantization-aware training (QAT)	Fine-tune/train the quantized model after training.	High	Low	Low	High
Knowledge Distillation (KD)	Change the model architecture and remove/fuse a few layers to reduce the size/computational costs	High	Low	Medium	
Pruning	Remove individual weights/rows/columns of weights matrices	High	Low	Medium	

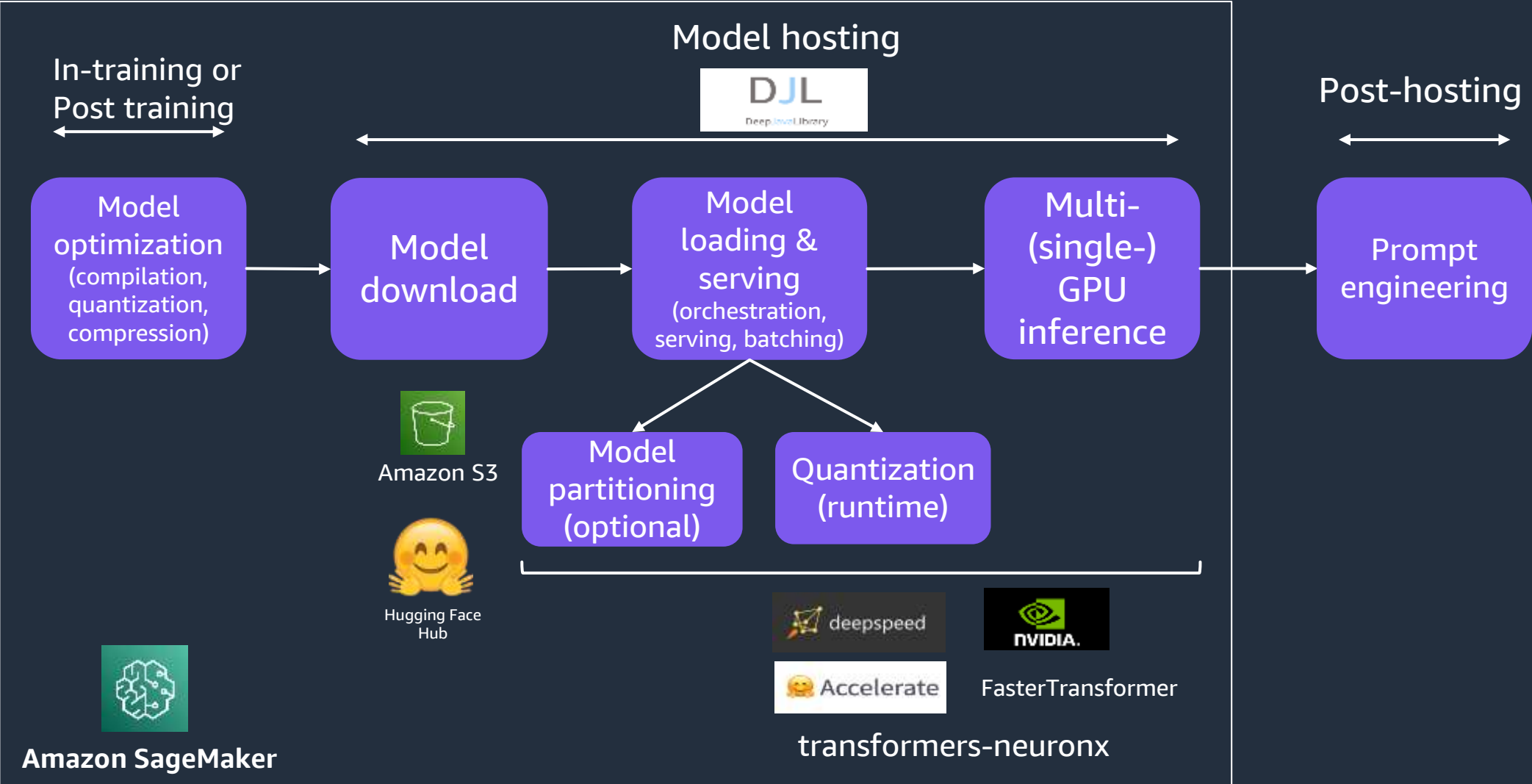
Large Model Hosting



Large Generative model inference optimization



Large model optimization and hosting pipeline on SageMaker



Sagemaker Large Model Inference Container

Large ML models
with 100 billion + parameters



Easily parallelize models across multiple GPUs to fit models into the instance and achieve low latency

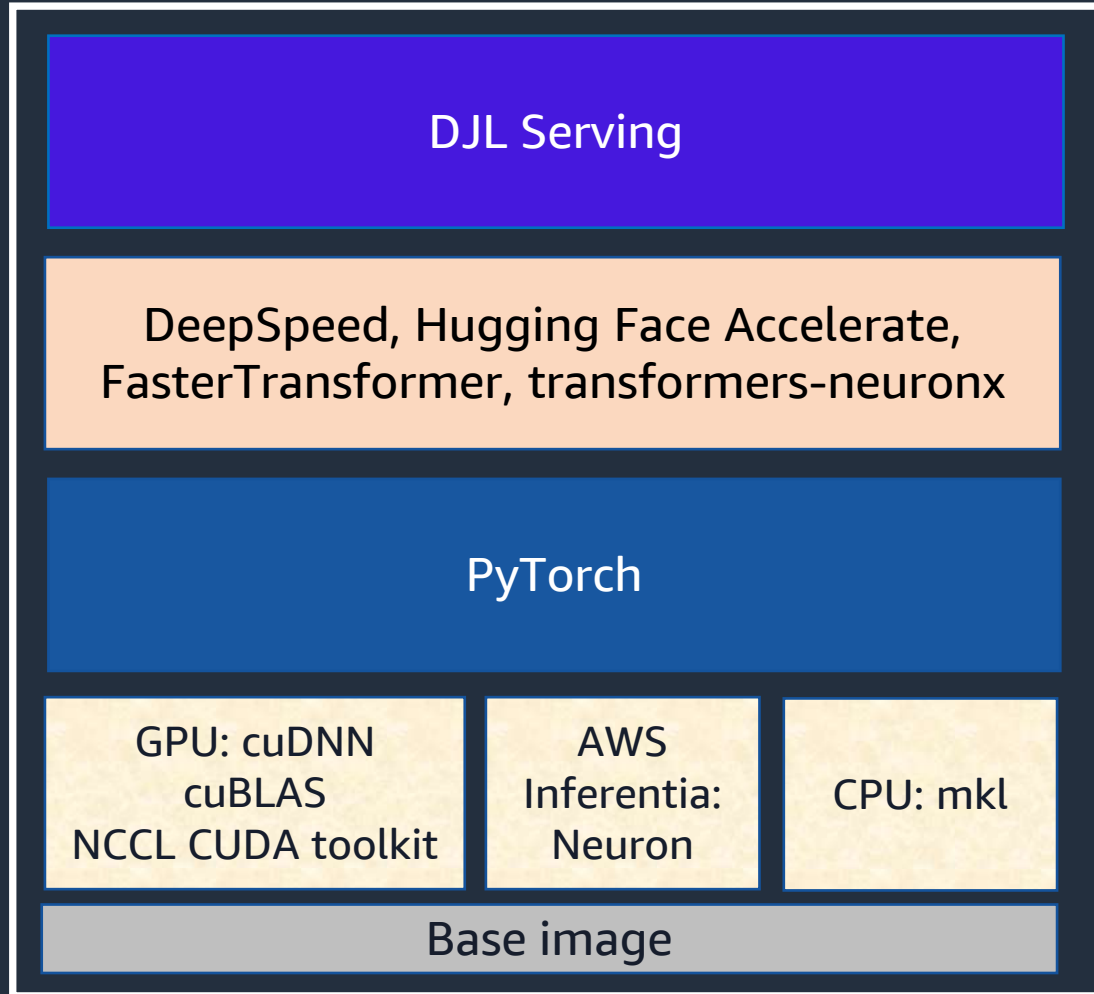


Deploy models on the most performant and cost-effective GPU-based instances or on AWS Inferentia



Leverage 500GB of Amazon EBS volume per endpoint

SageMaker Large Model Inference Container



Zero-code setup: [DeepSpeed](#), [StableDiffusion](#), and [Hugging Face](#), FasterTransformer, transformers-neuronx handlers

Model Server

Model server – Model servers handle an inference request end to end.

- They accept requests, invoke pre-processing and post-processing scripts, and respond to users.

SageMaker supports the following model servers:

[DJL-Serving](#) is an open-source, high-performance model server powered by [DJL](#). It takes multiple deep learning models or workflows, and makes them available through an HTTP endpoint.

Model parallelism and inference optimization libraries

- DeepSpeed Inference is an open-source inference optimization library.
- Includes model partitioning schemes for model parallelism with supported models, including many transformer models.
- It also has **optimized kernels for** popular models such as OPT, GPT, and BLOOM that can significantly improve inference latency.
- The version of DeepSpeed in the LMI DLCs is optimized and tested to work on SageMaker. It includes several enhancements, including support for BF16 precision models.

Model parallelism and inference optimization libraries

- FasterTransformer is an open source library from Nvidia that provides an accelerated engine for efficiently running transformer-based neural network inference.
- Designed to handle large models that require multiple GPUs and nodes in a distributed manner.
- The library includes an optimized version of the transformer block, which comprises both the encoder and decoder parts, enabling you to run the inference of full encoder-decoder architectures (T5, BERT, GPT et.)

Model parallelism and inference optimization libraries

Hugging Face Accelerate is an open-source model parallel inference library. It supports model parallelism for most models in the Hugging Face Transformers library.

Large model inference (LMI) container on Amazon SageMaker

Large ML models

with 100 billion + parameters



- ✓ Faster model download time using s5cmd
- ✓ Supported by AWS and open source
- ✓ Low-code/no-code deployment
- ✓ Native integration with Inferentia/Trainium
- ✓ SageMaker multi-model, multi-container, batch, and serverless
- ✓ Pre-built optimized model parallel frameworks including transformers-neuronx, DeepSpeed, Hugging Face Accelerate, and FasterTransformer
- ✓ Pre-built foundation software stack including PyTorch, NCCL, and MPI



Model Serving Framework – key considerations

	Hugging Face Accelerate	DeepSpeed	TensorRT	FasterTransformer
Approach	Runtime	Ahead of time & Runtime	Ahead of time	Ahead of time
Supported Model types	All Hugging face	Most of the Language Transformer models	GPT-2, Computer Vision models, general purpose models	Most of the Language Transformer models
PTQ accuracy	High	High	Moderate	High
Quantized inference	FP32, FP16, INT8	FP32, FP16, BF16, Simulated INT8	Yes	FP32, FP16, BF16
Inference speed	Slower	Fast	Fast	Fast
Frameworks supported	PyTorch	PyTorch	PyTorch, TensorFlow, ONNX	PyTorch
Optimized kernels available	Yes	Yes	Generated on-the-fly at runtime	Yes



LLM handlers

No Code: serving.properties

DeepSpeed with BF16 and S5cmd

```
engine=DeepSpeed
option.entryPoint=djl_python.deepspeed
option.s3url=s3://djl-llm/gpt-j-6b/
option.task=text-generation
option.tensor_parallel_degree=2
option.dtype=bf16
```

StableDiffusion with Neuron

```
engine=Python
option.entryPoint=djl_python.transformers-neuronx
option.use_stable_diffusion=True
option.model_id={{s3url}}
option.tensor_parallel_degree=2
option.dtype=bf16
```

HuggingFace with Accelerate and bitsandbytes

```
engine=Python
option.entryPoint=djl_python.huggingface
option.task=text-generation
option.model_id=bigscience/bloom-7b1
option.tensor_parallel_degree=4
option.load_in_8bit=TRUE
option.device_map=auto
```



Performance best practices



Performance best practices

1. Use dynamic batching to increase the throughput
2. Use tensor parallel over pipeline parallel sharding to improve hardware efficiency
3. Increase the copies of model to maximize the use of accelerator's memory
4. Tune number of shards to avoid running out of memory
5. Quantize your models to compress the model
6. Use AWS Inferentia2/AWS Trainium for supported model architectures
7. Select the right model sharding framework to reduce response times
8. Use s5cmd with LMI container to improve the container startup time
9. Follow the autoscaling best practices for SageMaker Inference endpoint as described in [this blog post](#)
10. Fine-tune the ML framework specific parameters (threads, workers, memory arena, etc.)
11. Use binary format for larger payloads (images, etc.)
12. Get the best from your model using different decoding techniques, including prompt engineering
13. Use SageMaker Inference Recommender to benchmark your inference workloads and choose the right hardware



Cost



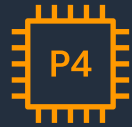
- Infrastructure costs
- Operations and maintenance costs

Infrastructure cost



Inferentia and Trainium: 2.3x throughput with 70% lower cost compared to GPU instances

Choose the right instances



GPU: G5 series, P4 series



EC2 Auto Scaling: Provision instances dynamically to meet traffic pattern; auto scales in a few minutes

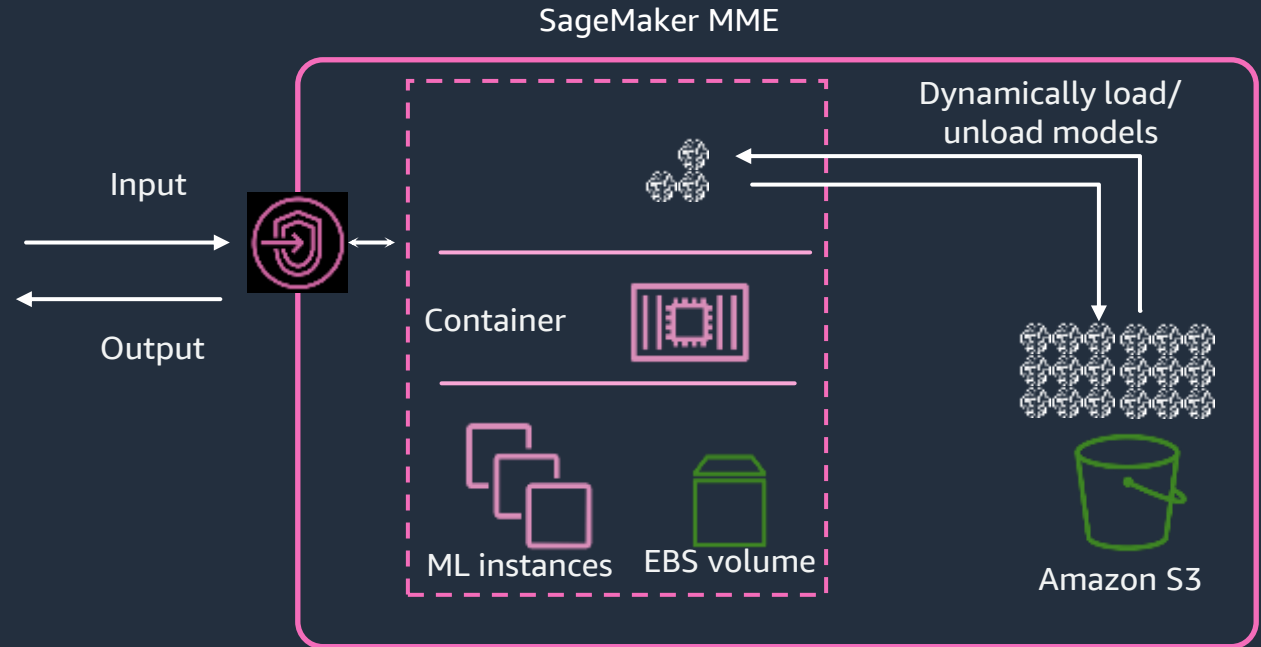
SageMaker multi-model endpoints (MME)



Deploy thousands of models to a single endpoint



Save 90% of cost deploying personalization models



Why run Generative Model Inference on Amazon SageMaker?

Infrastructure Management & Monitoring

Optimized deep learning & ML framework containers

Serving stack installation

Request routing +
Smart routing (multi-model endpoints)

Storage configuration

VPC, NAT/IGW provisioning

Model Release Management

A/B testing

Blue/green deployments

Canary & Linear traffic shifting

Update endpoints w/o availability loss

Auto-rollback protection

Security & Compliance

VPC endpoint support

Authentication (Sigv4) / Authorization

Secure connection (TLS 1.2)

Support for CMK & KMS

Security compliance validation

ML-Specific Capabilities

High level Python SDK, AWS SDKs,
CLIs, APIs

Multiple deployment modes

Inference pipelines

Pre/post data processing

Request & response capturing

Model version management

Model monitoring

Inference load testing

Explainability

Lineage tracking

Model registry

Model caching
(with multi-model endpoints)

Generative Model hosting

Faster model downloads

Large model Partitioning across GPUs
(DeepSpeed, Hugging Face Accelerate)

AWS Inferentia2/Trainium support

SageMaker compatible pre-built
containers

Low-code/No-code setup using
JumpStart

High Availability

Autoscaling & Continuous health
checks

Automatic monitoring and patching
of underlying instances

Automatic storage monitoring

Automatic bad instance replacement

Automatic multi-AZ provisioning
(provides 99.95% SLA for real-time inference)

Cost Optimization Tooling

Multi-model endpoints

Multi-container endpoints

Instance rightsizing

Serverless Inference (zero cost if idle)

Model compilation



Summary – Hosting LLMs

1. **Choose the LLM** – large is not necessarily right – Sagemaker Jumpstart, Model zoo, BYO-open source
2. **Choose the right instance type** and size (cpu, gpu, memory - p3 12x, p4d 12x, g4, g5 16x, Inf1, Inf2- Inferentia instances)
3. Choose and configure the **Sagemaker Model Serving, Inference Container** (DJL Serving, DeepSpeed, compatible versions framework, etc..) – check the Amazon Elastic Container Registry
4. Configure the data Volume size – 500GB+
5. Deploy model - Sagemaker Model endpoint type (Real time, Batch, etc..)
6. Explore possibility of multi-model endpoint
7. Capture and Monitor logs – CloudWatch



References

1. <https://docs.aws.amazon.com/sagemaker/latest/dg/large-model-inference.html>
2. Blog: Deploy BLOOM-176B and OPT-30B on Amazon SageMaker with large model inference Deep Learning Containers and DeepSpeed
3. Blog: Deploy large models on Amazon SageMaker using DJL Serving and DeepSpeed model parallel inference
4. Example notebooks: AWS SageMaker examples GitHub repository.





Thank you!

Sudhanshu Hate
HiSuds@amazon.com

Deep Learning Container Inference images

DLC URI	Libraries	Tuning parameters
763104351884.dkr.ecr.region.amazonaws.com/djl-inference:0.23.0-fastertransformer5.3.0-cu118	PyTorch 2.0.1	https://github.com/deepjavalibrary/djl-serving/blob/0.23.0-dlc/serving/docs/configurations_large_model_inference_containers.md#fastertransformer-doc
	DJL Serving 0.23.0	
	FasterTransformer 5.3.0	
	Hugging Face Accelerate 0.20.3	
	Hugging Face Transformers 4.30.1	
763104351884.dkr.ecr.region.amazonaws.com/djl-inference:0.23.0-deepspeed0.9.5-cu118	PyTorch 2.0.1	https://github.com/deepjavalibrary/djl-serving/blob/0.23.0-dlc/serving/docs/configurations_large_model_inference_containers.md#deepspeed-doc
	DJL Serving 0.23.0	
	DeepSpeed 0.9.5	
	Hugging Face Accelerate 0.20.3	
	Hugging Face Transformers 4.30.2	
763104351884.dkr.ecr.region.amazonaws.com/djl-inference:0.23.0-neuronx-sdk2.12.0	PyTorch 1.13.1	https://github.com/deepjavalibrary/djl-serving/blob/0.23.0-dlc/serving/docs/configurations_large_model_inference_containers.md#transformers-neuronx-doc
	DJL Serving 0.23.0	
	TransformersNeuronX 0.4.60	
	AWS Neuron SDK 2.12.0	
	NeuronX Distributed 0.1.0	
763104351884.dkr.ecr.region.amazonaws.com/djl-inference:0.21.0-fastertransformer5.3.0-cu117	PyTorch 1.12.1	https://github.com/deepjavalibrary/djl-serving/blob/0.21.0-dlc/serving/docs/configurations.md
	DJL Serving 0.21.0	
	FasterTransformer 5.3.0	
	Hugging Face Accelerate 0.15.0	
	Hugging Face Transformers 4.25.1	

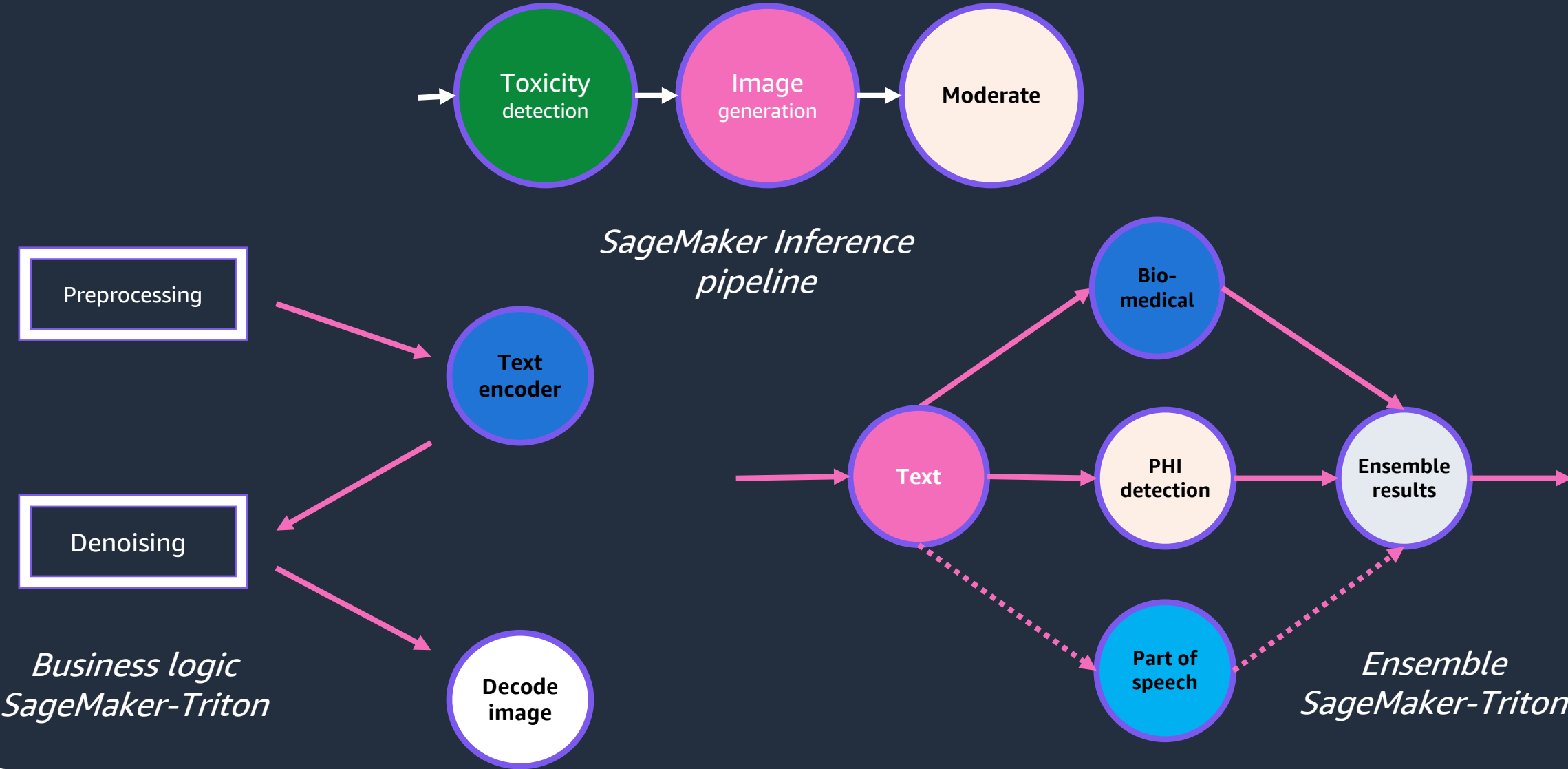
Deep Learning Container Inference images

DLC URI	Libraries	Tuning parameters
763104351884.dkr.ecr.region.amazonaws.com/djl-inference:0.19.0-deepspeed0.7.3-cu117	PyTorch 1.12.1	https://github.com/deepjavalibrary/djl-serving/blob/0.19.0-dlc/serving/docs/configurations.md
	DJL Serving 0.21.0	
	DeepSpeed 0.8.0	
	Hugging Face Accelerate 0.16.0	
	Hugging Face Transformers 4.26.0	
763104351884.dkr.ecr.region.amazonaws.com/djl-inference:0.20.0-deepspeed0.7.5-cu116	PyTorch 1.12.1	https://github.com/deepjavalibrary/djl-serving/blob/0.20.0-dlc/serving/docs/configurations.md
	DJL Serving 0.20.0	
	DeepSpeed 0.7.5	
	Hugging Face Accelerate 0.13.2	
	Hugging Face Transformers 4.23.1	
763104351884.dkr.ecr.region.amazonaws.com/djl-inference:0.19.0-deepspeed0.7.3-cu113	Hugging Face Diffusers 0.7.2	https://github.com/deepjavalibrary/djl-serving/blob/0.19.0-dlc/serving/docs/configurations.md
	PyTorch 1.12.1	

Large model inference patterns with Amazon SageMaker

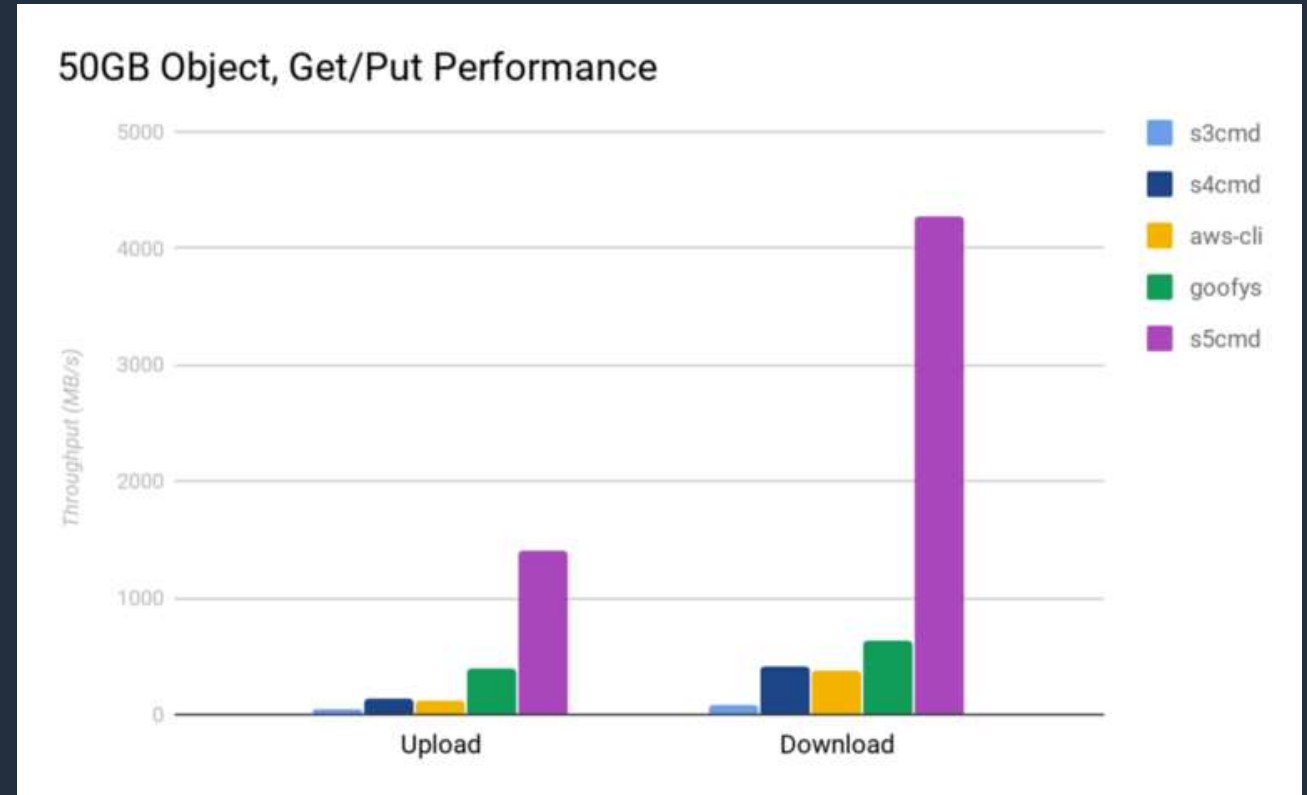


Inference workflows



Reduce model download time using LMI

- ~50% reduction in model download time
- Activate in LMI container with 1 line
 - In serving.properties, just add `option.s3url=<Your S3 path>`



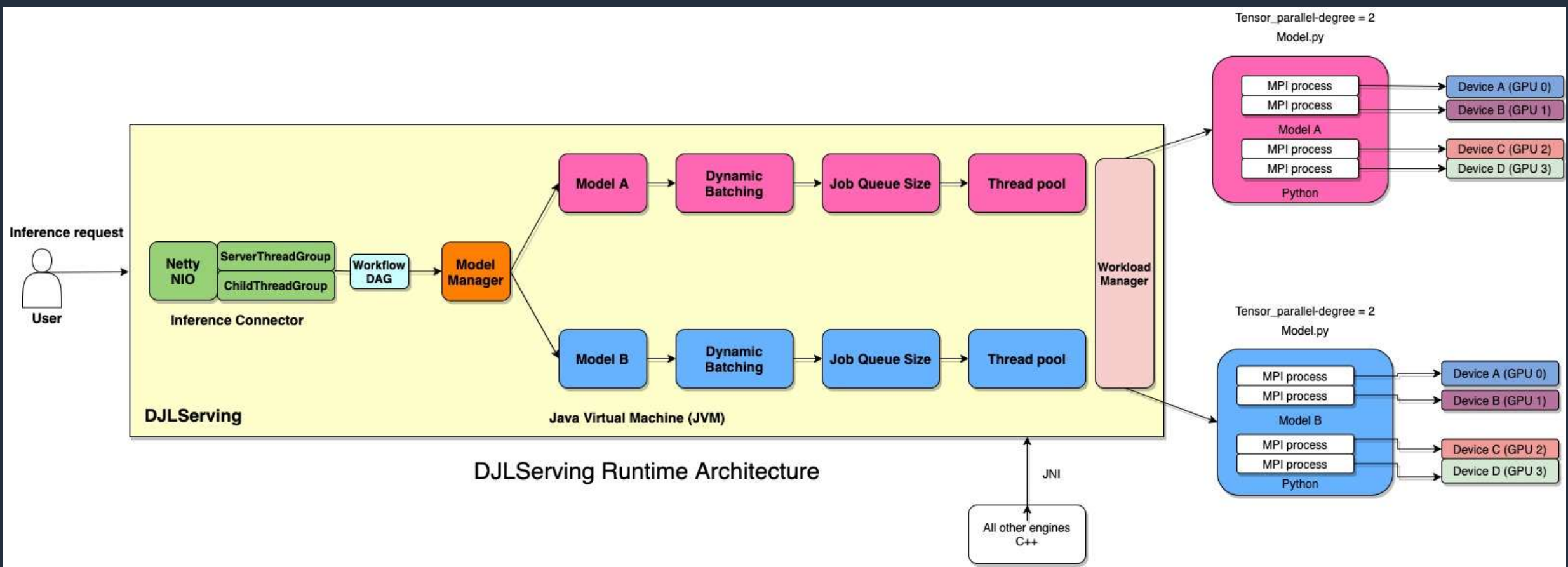
Appendix



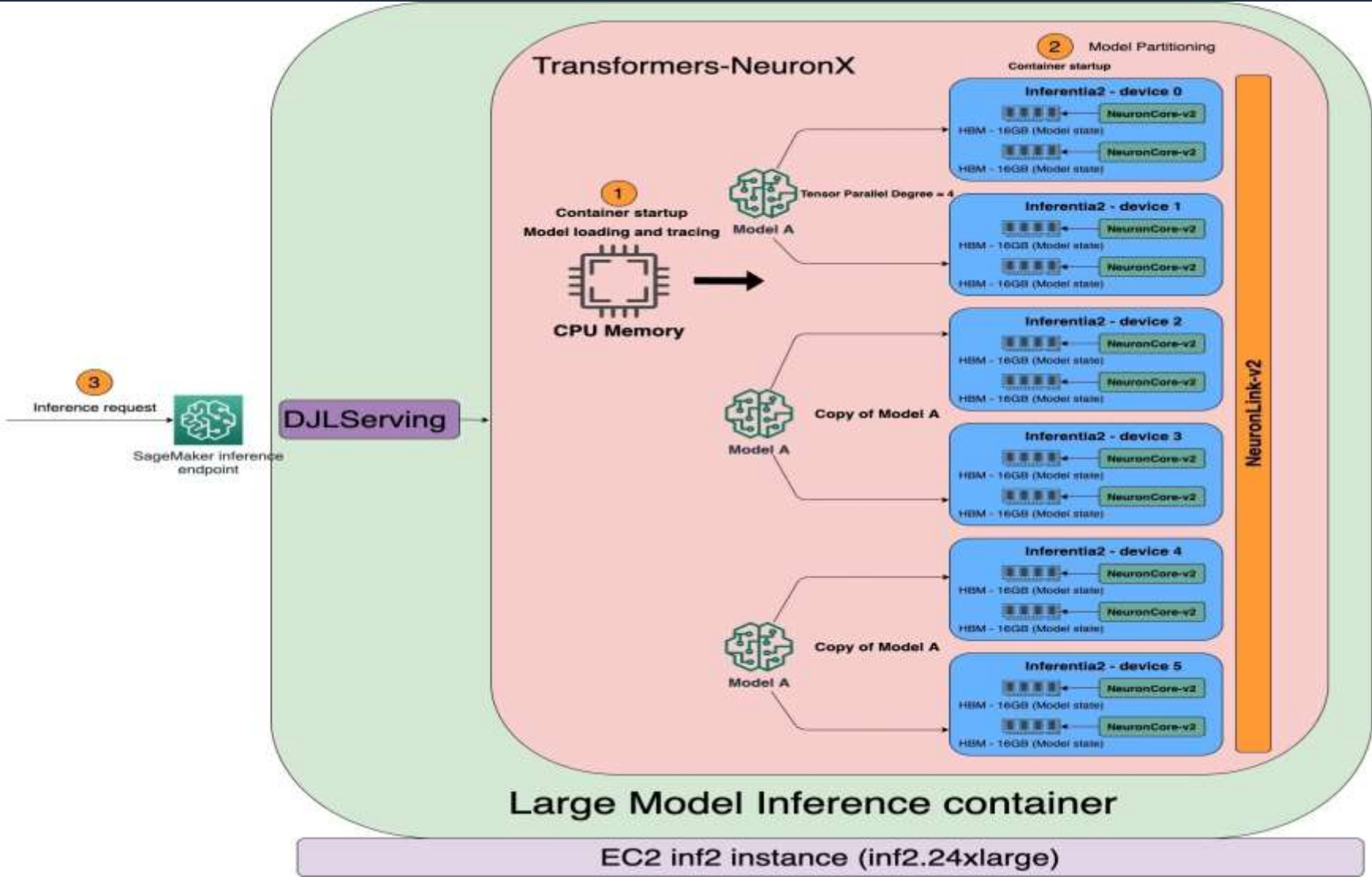
Large model inference container (LMI) – Deep dive



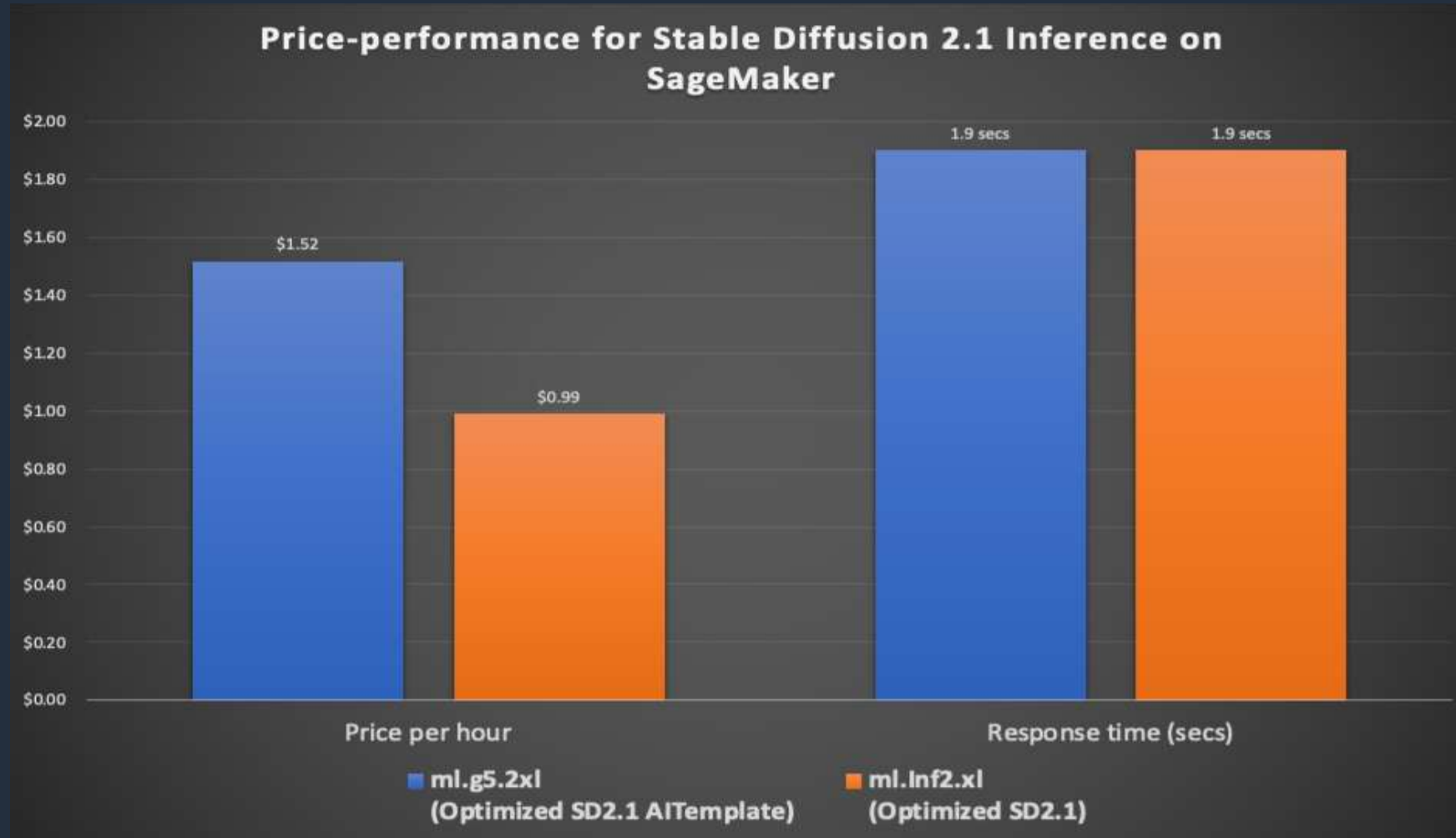
DJL Serving using DeepSpeed on SageMaker



Runtime architecture – transformers-neuronx

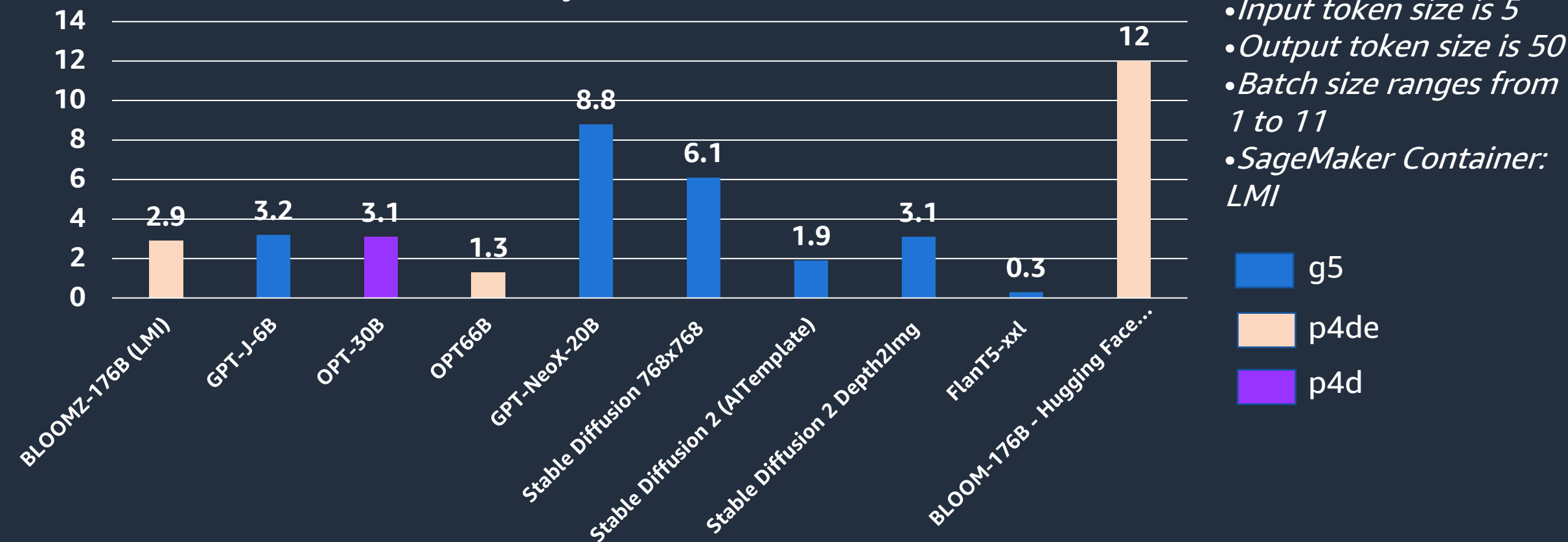


Benchmarking results for stable diffusion on SageMaker & AWS Inferentia2



Benchmarking results for large model inference on Amazon SageMaker

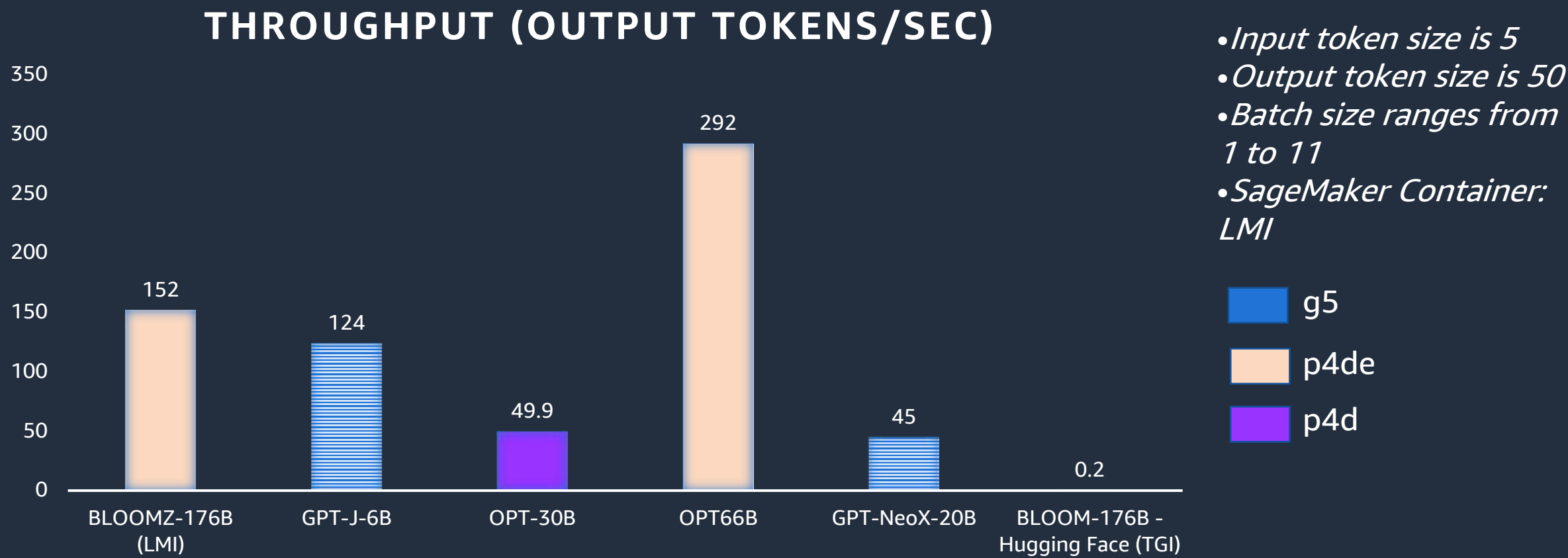
Latency P90 in secs



1] BLOOM-176B was tested on p4de.24xlarge (A100) with batch size 50, and int8 showed throughput of 350 tokens/sec.
[2] Stable Diffusion 2.1 model is 768x768 image generation with 50 denoising steps. All other values are default
[3] Stable Diffusion 2 Depth model using 512x512 image, DDIM scheduler with 512-base model, 62 denoising steps, strength of 0.8 (50 effective steps). All other values are default.
[4] TGI to [Hugging Face Text Generation Inference library](#)



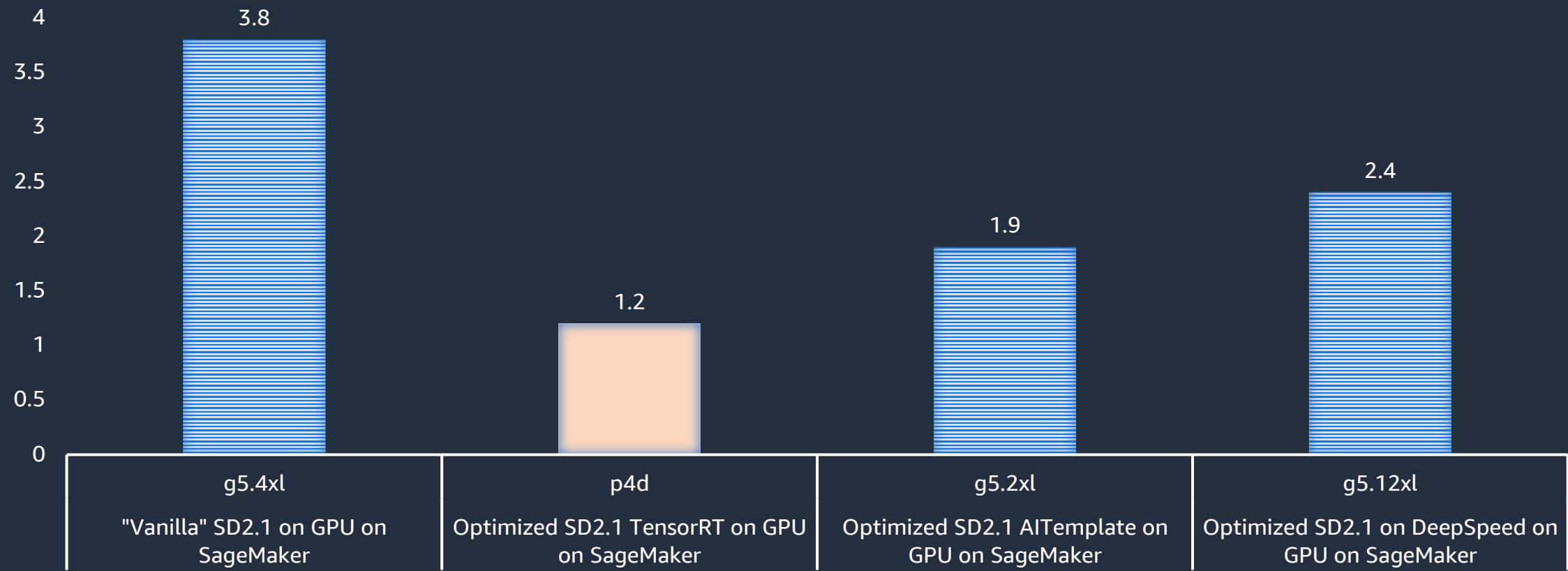
Benchmarking results for large model inference on Amazon SageMaker



1] BLOOM-176B was tested on p4de.24xlarge (A100) with batch size 50, and int8 showed throughput of 350 tokens/sec.
[2] Stable Diffusion 2.1 model is 768x768 image generation with 50 denoising steps. All other values are default
[3] Stable Diffusion 2 Depth model using 512x512 image, DDIM scheduler with 512-base model, 62 denoising steps, strength of 0.8 (50 effective steps). All other values are default.
[4] TGI to [Hugging Face Text Generation Inference library](#)

Benchmarking results for large model inference on Amazon SageMaker

LATENCY IN SECONDS



Large model inference container (LMI) – Deep dive

Resources

Hosting large generative AI models



Hosting **generative**
AI models workshop



Hosting BLOOM-176B
on **multiple** GPUs



Common model hosting
patterns on SageMaker



Deploy large models
using FasterTransformer



Resources



Forethought: Cost savings of 66% with SageMaker



Customize chatbots with **OpenChatKit** on SageMaker

